# Exit Ticket · reflect and retrieve

## Exit Ticket

Standard ASCII provides 128 character codes.

## Use key vocabulary from memory

1. Explain Character set in your own words.

2. Explain Code point in your own words.

3. Explain Encoding in your own words.

## Confidence and next step

How confident are you? Thumbs up / side / down

Which question would you improve after checking the classroom model?

Check answers after the lesson using the teacher-guided source models.





# Do Now · retrieval

## Character sets: ASCII & Unicode

Name: _____ Class: _____ Date: _____

Answer four retrieval questions before reviewing earlier learning.

### Q1

Which data type should store a telephone number beginning with 0? Explain briefly.

### Q2

Convert $41_{16}$ to denary and eight-bit binary.

### Q3

Apply AND 00001111 to 10100101. Explain which bits the mask preserves.

### Q4

In 6+4 two’s-complement floating-point format, subtract 010000 0010 from 011000 0010. Give a normalised result.

# New Learning · key ideas

The important knowledge from today's lesson. Read, annotate, and explain it.

### Core idea 1

Characters need agreed codes — A character set defines characters and their assigned numeric codes. An encoding specifies how those codes are represented as bytes.

### Core idea 2

Standard ASCII is limited — Seven bits provide $2^7 = 128$ codes for letters, digits, punctuation and control characters. — Eight-bit extensions can provide 256 values, but different extensions may map the extra codes differently.

### Core idea 3

Unicode supports a much larger repertoire — Unicode assigns code points to characters from many writing systems and symbols. Its first 128 code points match ASCII.

### Annotate the key visual

Explain which labels or features show the main process.





# You Do · extended practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

### Question 2

Question 5 · ORIGINAL ENCODING APPLICATION · [4]
A UTF-8 string contains 20 ASCII code points, three euro-sign code points and two emoji code points. The byte widths are 1, 3 and 4 respectively. Ignore metadata. Calculate bytes [2] and bits [1]. Explain why multiplying the number of code points by 2 bytes is not valid here. [1]

Check that you have used the technical terms and explained your decisions.

# You Do · independent practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

## Question 1

Question 1 · ADAPTED OCR PROGRAMMING TASK · [3]
Complete the three gaps to count uppercase ASCII letters. asc returns a character's code. A = 65 and Z = 90. The loop examines each character once.
CODE:

```
function countCapitals(text)
count = 0
for each character in text
code = asc(character)
if code >= 65 _____ code <= 90 then
count = _____
endif
next character
return _____
endfunction
```

Adapted from OCR June 2022 Q5(e)(i): simplified loop and changed gaps; classroom allocation of one mark per gap. OCR June 2022, Q5(e), page 24

Check that you have used the technical terms and explained your decisions.





# New Learning · applying the ideas

## Core idea 4

Use the encoding stated in storage questions — Storage depends on the number of code points and the number of bytes used for each one. Do not assume every Unicode character uses the same width. — CODE:
'0' in ASCII = 48 = $00110000_2$
integer 0 = 0

## Vocabulary · use the correct technical terms

Character set — A collection of characters with assigned codes.

Code point — A numeric identifier assigned to a character in Unicode.

Encoding — Rules for representing character codes as bytes.

ASCII — A standard seven-bit character code with 128 possible values.

## Check your understanding

Explain one key process in your own words using two of the terms above.

More animated examples are available in the original online lesson.

# I Do · worked example

## Problem

Encode the text “AB”

## Follow the worked method

1. Start with the character codes — CODE:
A = 65
B = 66 — ASCII assigns a numeric code to each character.

2. Write the 7-bit ASCII values — CODE:
A = 1000001
B = 1000010

3. Add a leading zero — CODE:
A → 01000001
B → 01000010

AB → 01000001 01000010 — The leading zero lets each 7-bit ASCII code sit inside an 8-bit byte; it does not expand the standard ASCII repertoire.

## Why this method works

Identify the decision, rule or invariant that makes the method correct.





# We Do · Try → Discuss → Improve

TRY independently in BLACK PEN; compare methods with a partner; improve your reasoning.

## Guided problem · TRY

Build the answer

## DISCUSS

Compare how you reached the answer. Where could someone make an error?

## IMPROVE

Add one precise correction or a stronger reason. Answers are not printed here.